UNIT 2 DECISION MAKING AND LOOPING

Conditional statements – if, if-else, nested if – Looping statements – for loop, while loop – break, continue, pass statements – Pattern generation – Problem solving using loops.

STATEMENT IN PYTHON

- A statement is an instruction that a Python interpreter can execute.
- Python statement ends with the token NEWLINE character.
- It means each line in a Python script is a statement.
- There are mainly four types of statements in Python,
    - Print statements
    - Assignment statements
    - Conditional statements
    - Looping statements

1.CONDITIONAL STATEMENTS

- Conditional statements in Python are used to make decisions in a program.
- They allow a program to execute different blocks of code depending on whether a condition is True or False.
- For example, if a student's mark is 50 or above, the program displays “Pass”; otherwise, it displays “Fail”.

TYPES OF CONDITIONAL STATEMENTS

- If statements (conditional)
- If-else statements (alternative)
- If-elif-else (chained conditional)
- Nested Conditional

1.1 If statements (conditional)

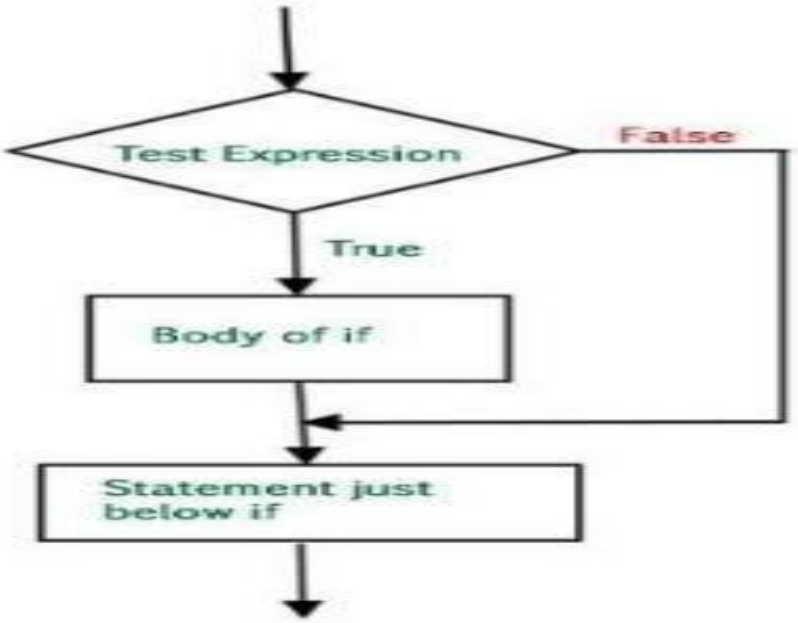
- The if statement contains a logical expression using which data is compared and a decision is made based on the result of the comparison.

- If statement is the simplest decision-making statement.
- It is used to decide whether a certain statement or block of statements will be executed or not.
- If a certain condition is true then a block of statement is executed otherwise not.

Syntax

If expression:
statement(s)

- If the Boolean expression evaluates to TRUE, then the block of statement(s) inside the if statement is executed.
- If Boolean expression evaluates to FALSE, then the first set of code after the end of the if statement(s) is executed.



python program to illustrate If statement

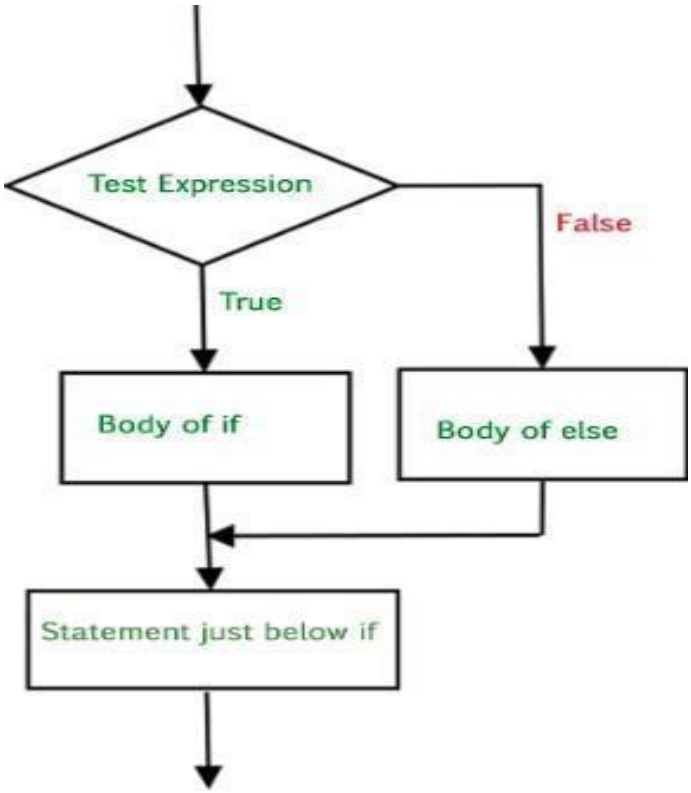
```
if (i > 15):
print ("10 is less than 15") print
("I am Not in if")
```

1.2 If – else statements (alternative)

- The if statement alone tells us that if a condition is true.
- It will execute a block of statements and if the condition is false, it won't.
- But if we want to do something else if the condition is false, here comes the else statement.
- We can use the else statement with if statement to execute a block of code when the condition is false.

Syntax

```
if (condition):
# Executes this block if
# condition is true
else:
# Executes this block if
# condition is false
```



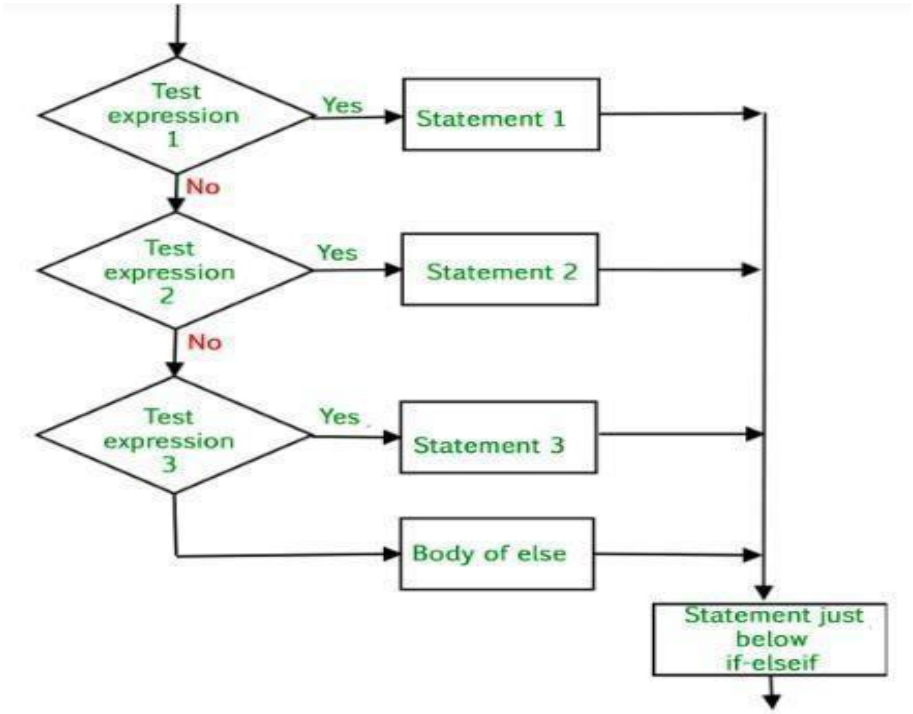
```
# python program to illustrate If else statement
i = 20;
if (i < 15):
print ("i is smaller than 15")
print ("i'm in if Block") else:
print ("i is greater than 15")
print ("i'm in else Block")
print ("i'm not in if and not in else Block")
```

1.3 If.. elif .. else (chained conditional)

- A user can decide among multiple options.
- The if statements are executed from the top down.
- As soon as one of the conditions controlling the if is true.
- The statement associated with that if is executed, and the rest of the ladder is bypassed.
- If none of the conditions is true, then the final else statement will be executed.

Syntax

```
if (condition):
statement
elif(condition):
statement
    .
    .
  else:
  statement
```




```
# Python program to illustrate if-elif-else ladder
if-elif
 i =20
 if (i == 10):
 print ("i is 10")
 elif (i == 15):
 print ("i is 15")
 elif (i == 20):
 print  ("i  is  20")
else:
   print ("i is not present")
```

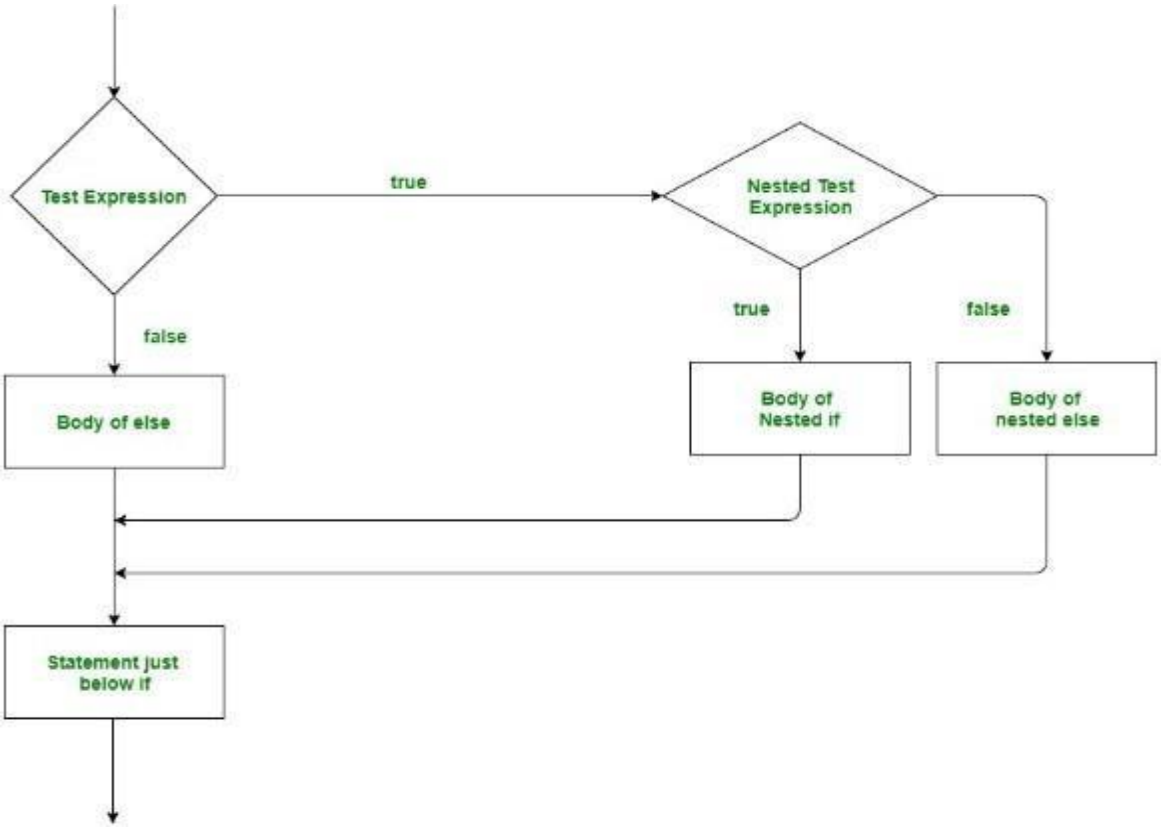
1.4 Nested if Conditional

- A nested if is an if statement that is the target of another if statement.
- Nested if statements means an if statement inside another if statement.
- Python allows us to nest if statements within if statements.
- we can place an if statement inside another if statements.

Syntax

```
if (condition1):
# Executes when condition1 is true

if (condition2):
# Executes when condition2 is true
# If Block is end here
```

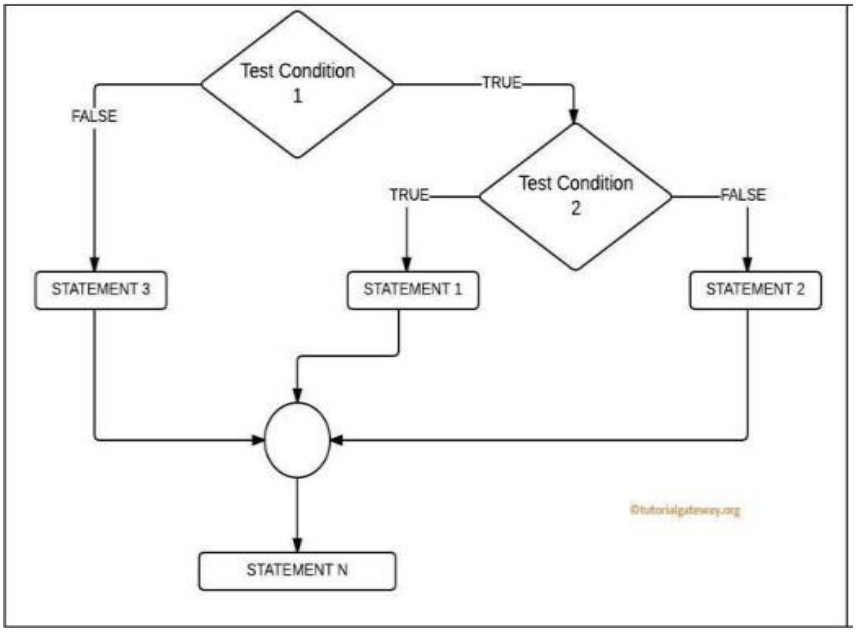


```
# python program to illustrate nested If statement

    i =10
    if (i == 10):
    # First if statement if (i < 15):
       print ("i is smaller than 15")
     # Nested - if statement
     # Will only be executed if statement above
    # it is true
        if (i < 12):
        print ("i is smaller than 12")
        else:
        print ("i is greater than 15")
```

ITERATION IN PYTHON

- A loop statement allows us to execute a statement.
- A group of statements multiple times.
- Repeated execution of a set of statements is called iteration.
- Programming languages provide various control structures.
- That allows for more complicated Types execution paths.



2.LOOPING STATEMENTS

- A looping statement in Python is used to execute a block of code.
- Repeatedly until a specified condition is satisfied.
- A sequence of items is completed.
- Loops help reduce repeated code.
- It makes programs simpler and more efficient.

TYPES OF LOOPING STATEMENTS

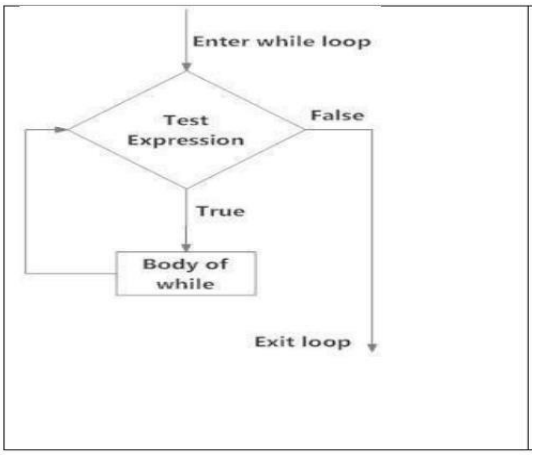
- While Loop
- For Loop
- Nested Loop

2.1 While Loop

- A while loop statement executes a block of statement again and again.
- Until the condition will occur FALSE.
- Repeats a statement or group of statements while a given condition is TRUE.
- It tests the condition before executing the loop body.
- This technique is known as Entry controlled loop.

Syntax

while expression:
statement(s)



Example

```
count = 0
while (count < 9):
        print("The count is:", count)
        count = count + 1
print("The End")
```

Output 1

```
The count is: 0
The count is: 1.....

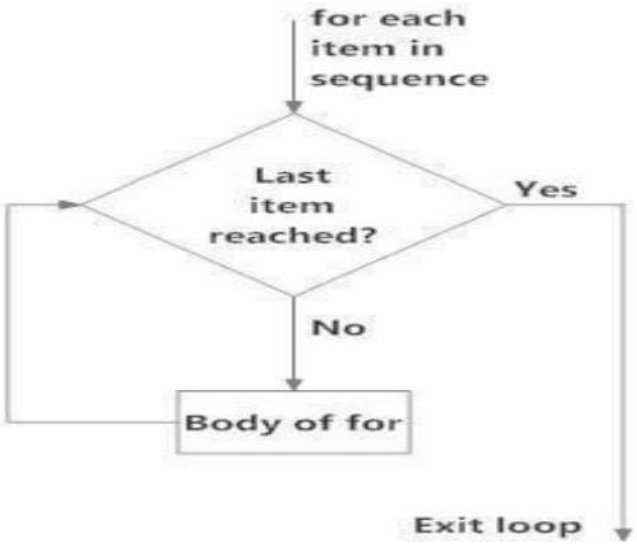
The count is: 8
The End
```

2.2 For Loop

- Executes a sequence of statements multiple times.
- Abbreviates the code that manages the loop variable.

Syntax

For iterating_var in sequence: statements(s)



Example for letter in 'Python'

```
print(“Current Letter :”, letter)
fruits =['banana', 'apple', 'mango']
for fruit in fruits:
 # Second Example
 print(”Current fruit :”, fruit)
 print("The End")
```

2.3 Nested Loops

- Python programming language allows using one loop inside another loop.
- Following section shows few examples to illustrate the concept.
- You can use one or more loop inside any another while, for or do...while loop.

Syntax - nested for loop

for iterating_var in sequence
statements(s)

Syntax - nested while loop

while expression

statement(s)

Example

```
# Find the prime numbers from 2 to 100:

i = 2

While (i < 100):

j = 2
While (j <= (i/j)):
 if not(i%j):
break
 j = j + 1 if (j > i/j) :

print ( i, " is prime")

i = i + 1

print ("The End")
```

Output

```
2 is prime
3 is prime

89 is prime

97 is prime
```

3.UNCONDITIONAL LOOPING STATEMENTS TYPES

- Break Statement
- Continue Statement
- Pass Statement

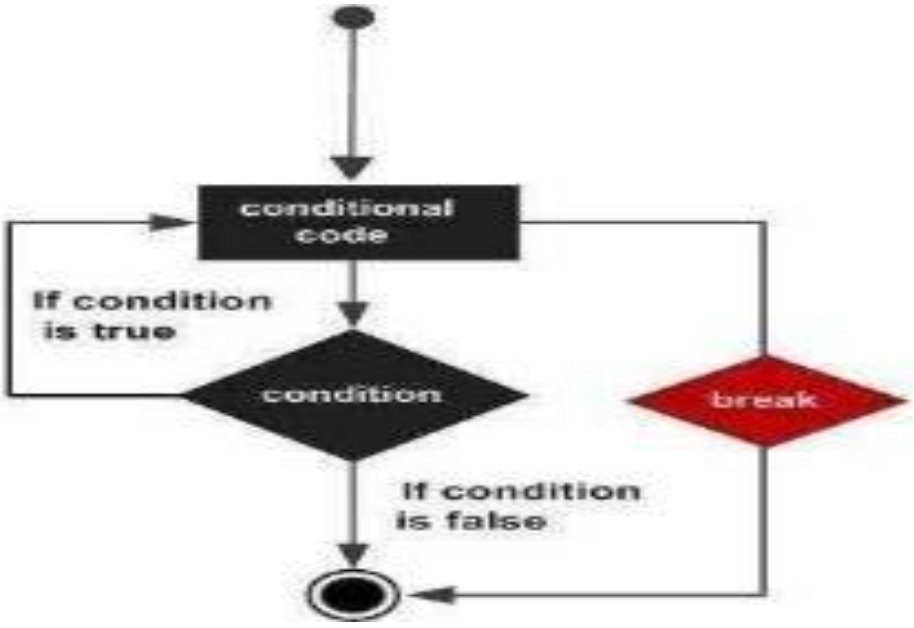
3.1 Break Statement

- Terminates the loop statement and transfers execution.
- Statement immediately following the loop.
- The most common use for break is when some external condition.
- It triggered requiring a hasty exit from a loop.
- The break statement can be used in both while and for loops.
- If you are using nested loops, the break statement stops the execution of the innermost loop.
- Start executing the next line of code after the block.

Syntax

Break

Flow chart



Example

```
for letter in 'Python':
# First Example if letter == 'h':
break
print(“Current Letter :”, letter)
var = 10
 # Second Example while var > 0:
print(“Current variable value :”, var)
var = var -1
if var == 5:
break
print( "The End")
```

Output

```
Current Letter : P
Current Letter : y
Current Letter : t
Current variable value : 10
Current variable value : 9
Current variable value : 8
Current variable value : 7
Current variable value : 6
```

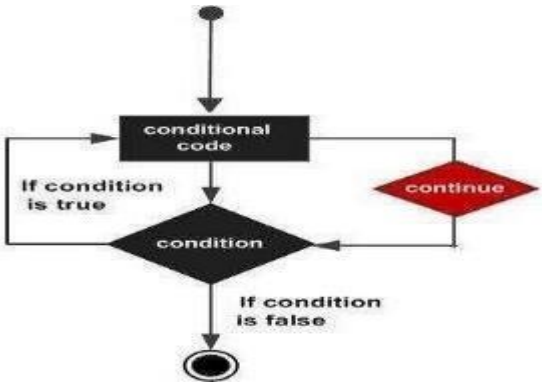
3.2 Continue Statement

- It returns the control to the beginning of the while loop.
- The continue statement rejects all the remaining statements.
- Current iteration of the loop and moves the control back to the top of the loop.
- The continue statement can be used in both while and for loops.

Syntax

Continue

Flowchart



Example

```
for letter in 'Python':
# First Example if letter == 'h':
pass
print(“This is pass block”)
print(“Current Letter :”, letter)
print("The End")
```

Output

```
Current Letter : P
Current Letter : y
Current Letter : t  This is pass block
Current Letter : h
Current Letter : o
Current Letter : n
```

3.3 The pass statement

- Pass is a null operation.
- Nothing happens when it executes.
- It is useful as a placeholder when a statement is required syntactically.
- But no code needs to be executed.

Syntax

```
# create a function def
  fun1(arg):pass
# a function that does nothing (yet)
```

Program

```
for i in range(1, 6):
 pass
 print("Loop completed")
```

Output

Loop completed

Explanation

- Range(1, 6) generates numbers from 1 to 5.
- The pass statement performs no action during each iteration.
- After the loop finishes, the program prints "Loop completed".

4.PATTERN GENERATION IN PYTHON

1.Define Pattern Generation

- Pattern generation in Python is the process of printing shapes, symbols, numbers.
- Its characters in a particular arrangement using loops.
- Patterns are commonly created using for loops, while loops, and the print() function.
- Python patterns help students understand loops, nested loops, and logical thinking.

2. Types of Pattern Generation

- Star patterns
- Number patterns
- Alphabet patterns
- Pyramid patterns
- Inverted patterns

2.1 Star Pattern Programs

Example 1: Right-Angled Triangle

program

```
rows = 5
for i in range(1, rows + 1):
for j in range(i):
print("*", end=" ")
print()
```

Output

```
*
* *
* * *
* * * *
* * * *
```

Explanation

- rows = 5 specifies the number of rows.
- The outer loop controls the rows.
- The inner loop prints stars in each row.
- end=" " prints stars on the same line.
- print() moves to the next line.

Example 2: Inverted Triangle

program

```
rows = 5
for i in range(rows, 0, -1):
for j in range(i):
 print("*", end=" ")
  print()
```

Output

```
* * * * *
* * * *
* * *
* *
*
```

Example 3: Pyramid Pattern

program

```
rows = 5
for i in range(1, rows + 1):
print(" " * (rows - i) + "* " * i)
```

Output

```
*
* *
* * *
* * * *
* * * * *
```

4. Number Pattern

Example 4: Increasing Number Triangle

program

```
rows = 5
for i in range(1, rows + 1):
for j in range(1, i + 1):
print(j, end=" ")
 print()
```

Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Example 5: Repeated Number Pattern

program

```
rows = 5
for i in range(1, rows + 1):
 for j in range(i):
 print(i, end=" ")
  print()
```

Output

1

2 2

3 3 3

4 4 4 4

5 5 5 5 5

Example 6: Alphabet Triangle

Python program

```
rows = 5
for i in range(1, rows + 1):
for j in range(i):
print(chr(65 + j), end=" ")
print()
```

Output

A

A B

A B C

A B C D

A B C D E

7. Applications of Pattern Generation

- Understanding nested loops.
- Improving logical thinking and problem-solving skills.
- Practising loop control and iteration.
- Preparing for Python practical examinations and programming interviews.
- Learning how to format and display output.

5.PROBLEM SOLVING USING LOOPS

Example: Sum of First 10 Natural Numbers.

Problem: Write a Python program to calculate the sum of the first 10 natural numbers using a loop.

Program

```
total = 0
for i in range(1, 11):
total = total + i
print("Sum =", total)
```

Output

Sum = 55

Explanation

- total = 0 initializes the sum.
- The for loop iterates from 1 to 10.
- Each number is added to total.
- Finally, the sum is displayed.